漫步数码时代，

铸就精彩人生，

程序员修炼之路，

从精雕算法开始……

第3章 动态规划

（Dynamic Programming）

（一）

主讲人：周正深

3.1 动态规划的设计思想

3.2 动态规划的设计要素

3.3 动态规划算法的典型应用

3.3.1 投资问题

3.3.2 背包问题

3.3.3 最长公共子序列LCS

3.3.4 图像压缩

3.3.5 最大子段和

3.3.6 最优二分检索树

3.3.7 生物信息学中的动态规划算法

3.1 动态规划的设计思想

起终点集已知的最短路径问题

例3.1 求从起点到终点的最短路径

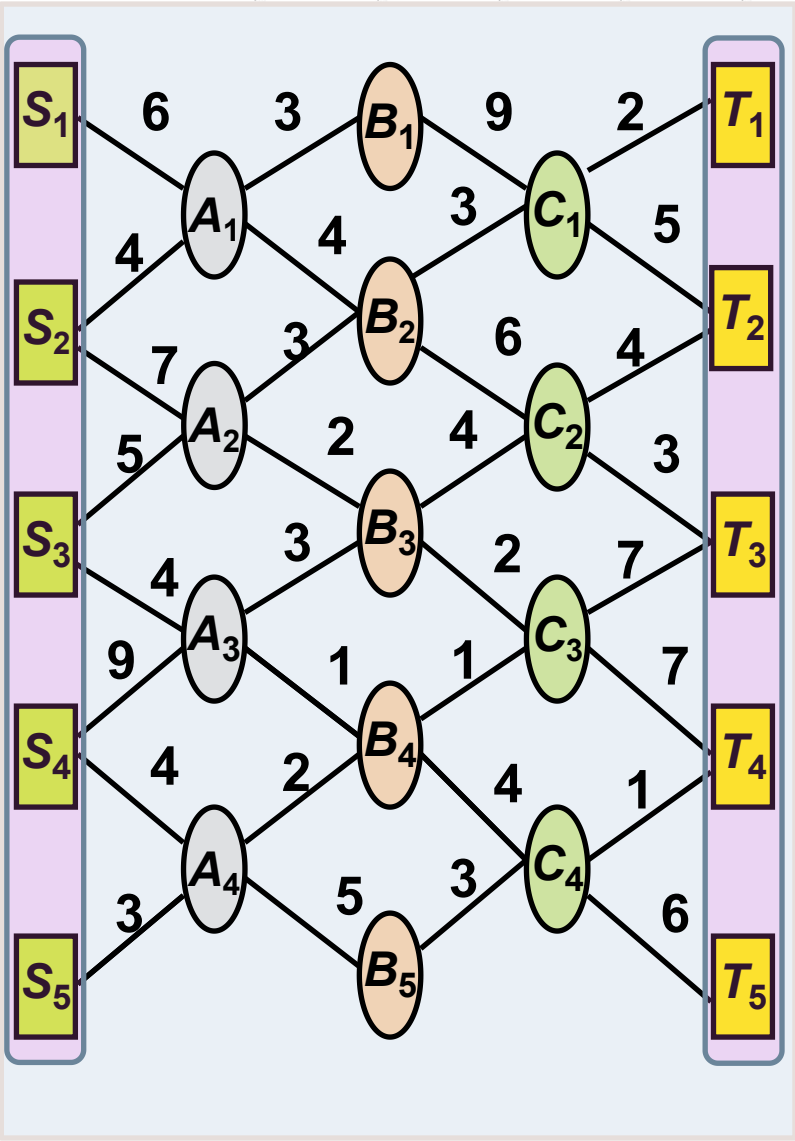
输入：起点集合$\{S_1, S_2, \cdots, S_n\}$，

终点集合$\{T_1, T_2, \cdots, T_m\}$，

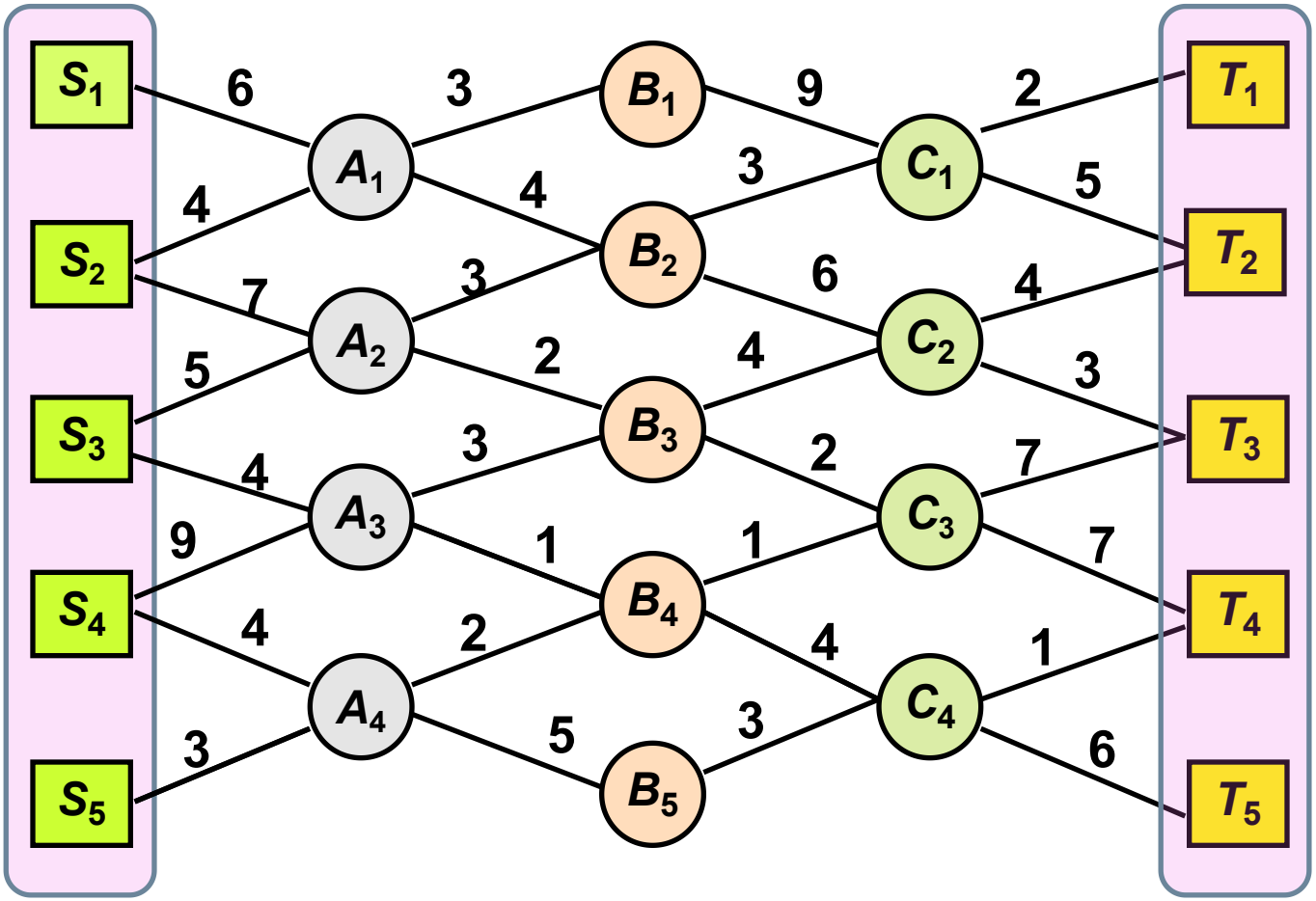
中间结点集，

边集E，对于任意边e有长度

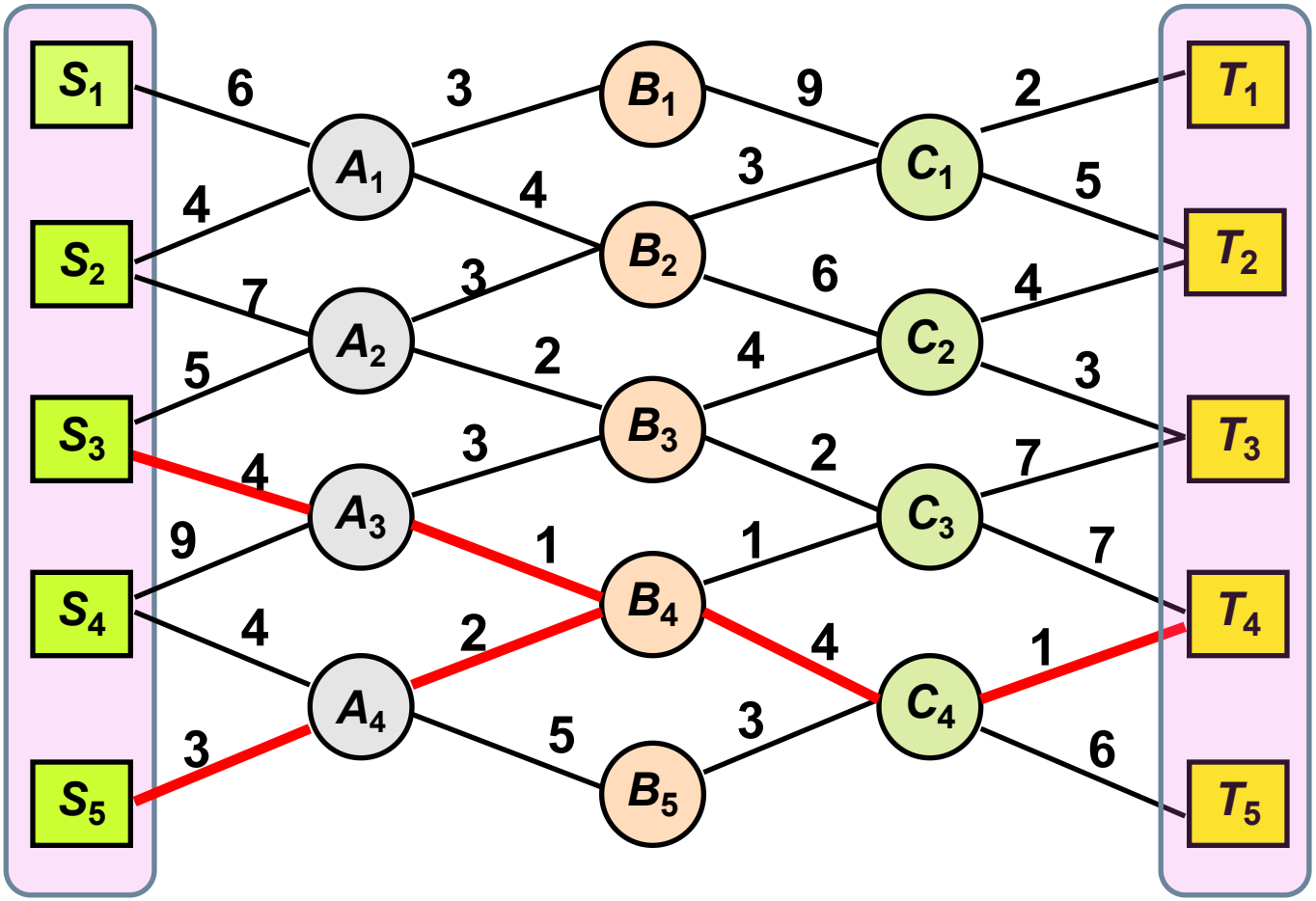
输出：起点到终点的最短路径



例3.1 求从始点到终点的最短路径



1. 蛮力算法(穷举)　求出所有路径，再比较找最小的。



蛮力算法复杂度分析：

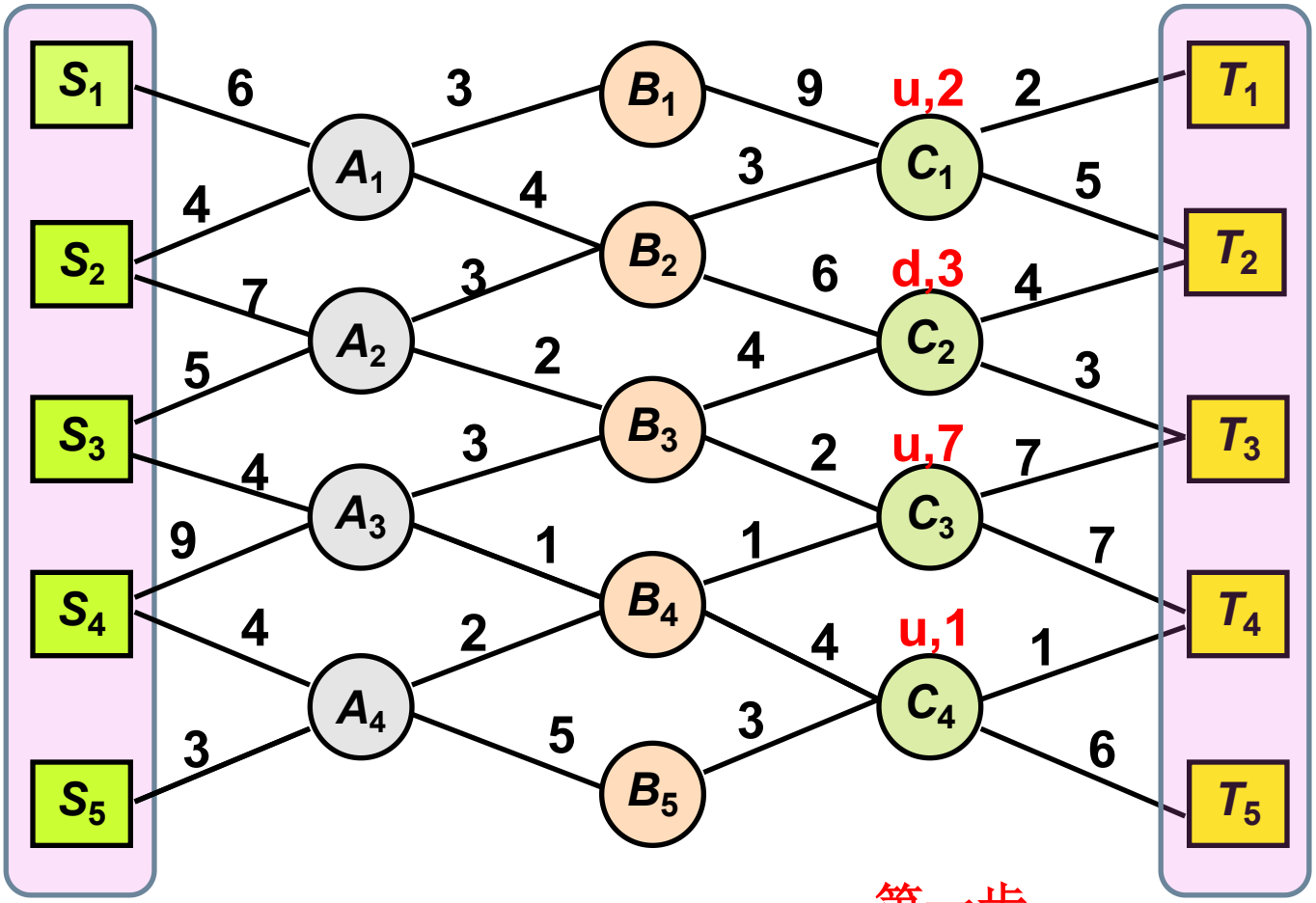
假设经过的网络层数是k，起点个数是n，那么路径条数约为$n2^k$, 每条加法$k-1$次。

所以，算法复杂度为$O(nk2^k)$

2. 动态规划算法　多阶段决策过程。

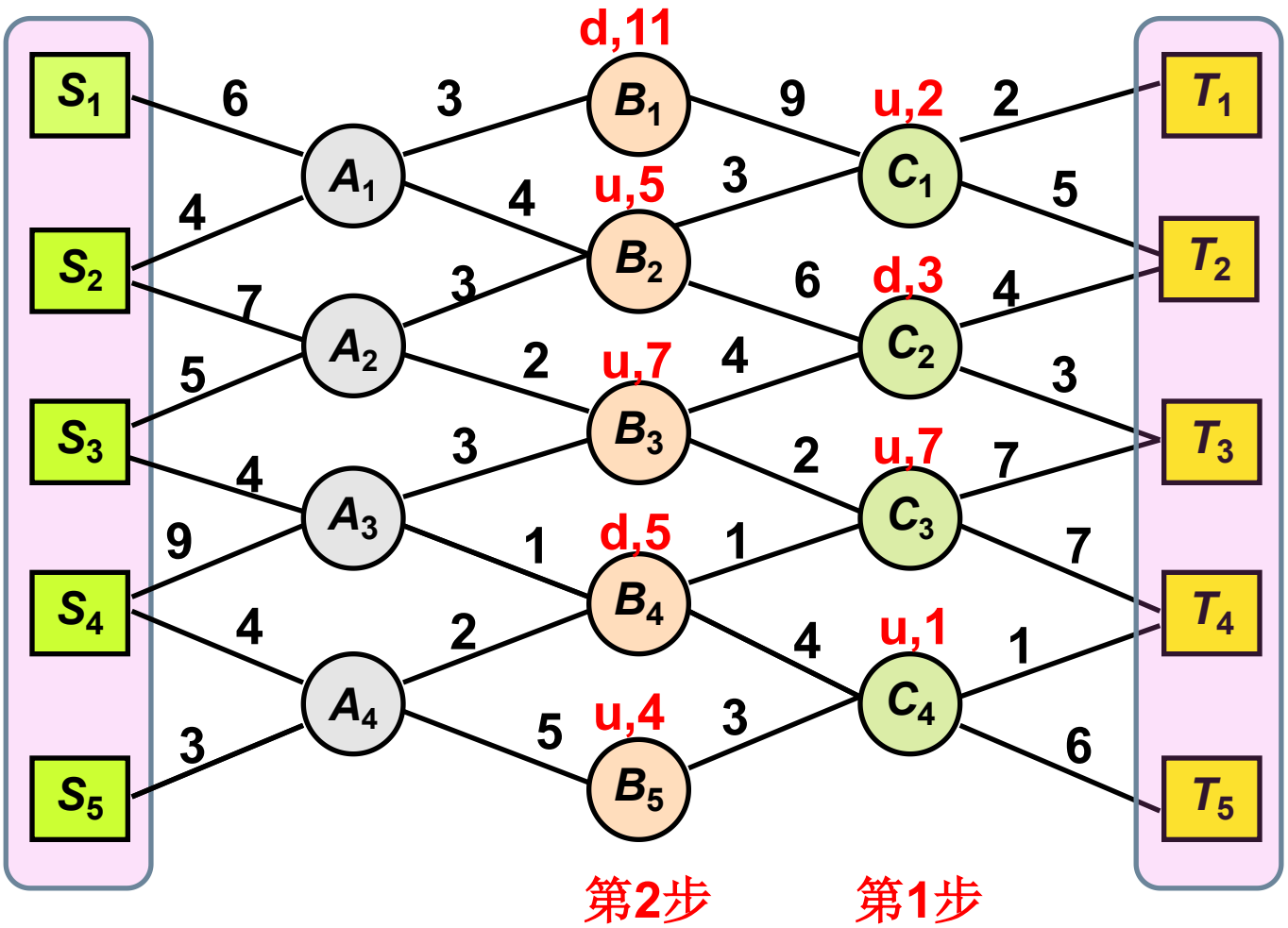
每步求解的问题是后面阶段求解问题的子问题。每步决策将依赖于以前步骤的决策结果。

动态规划算法求解过程：第1步(down，up)

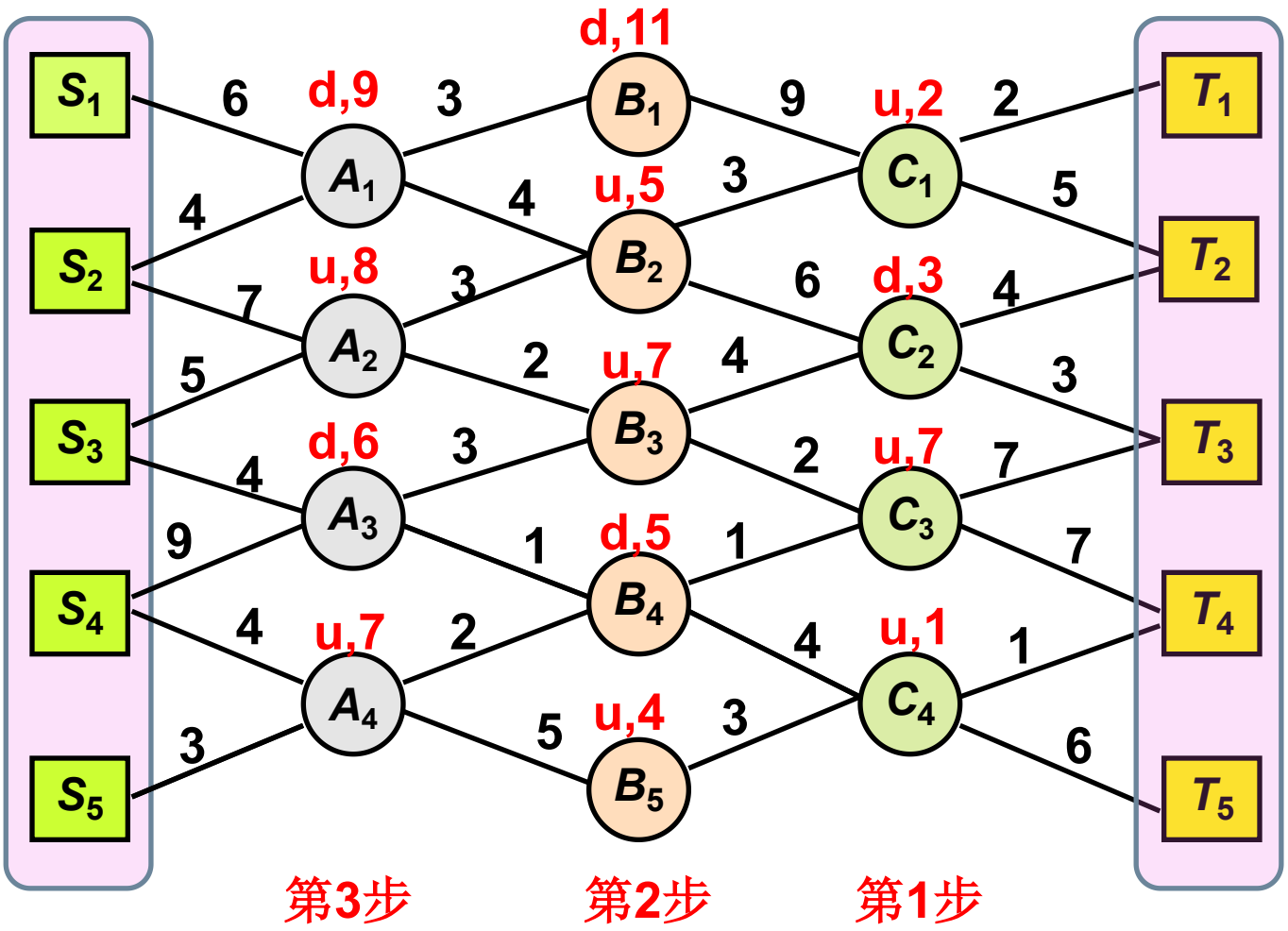


第一步

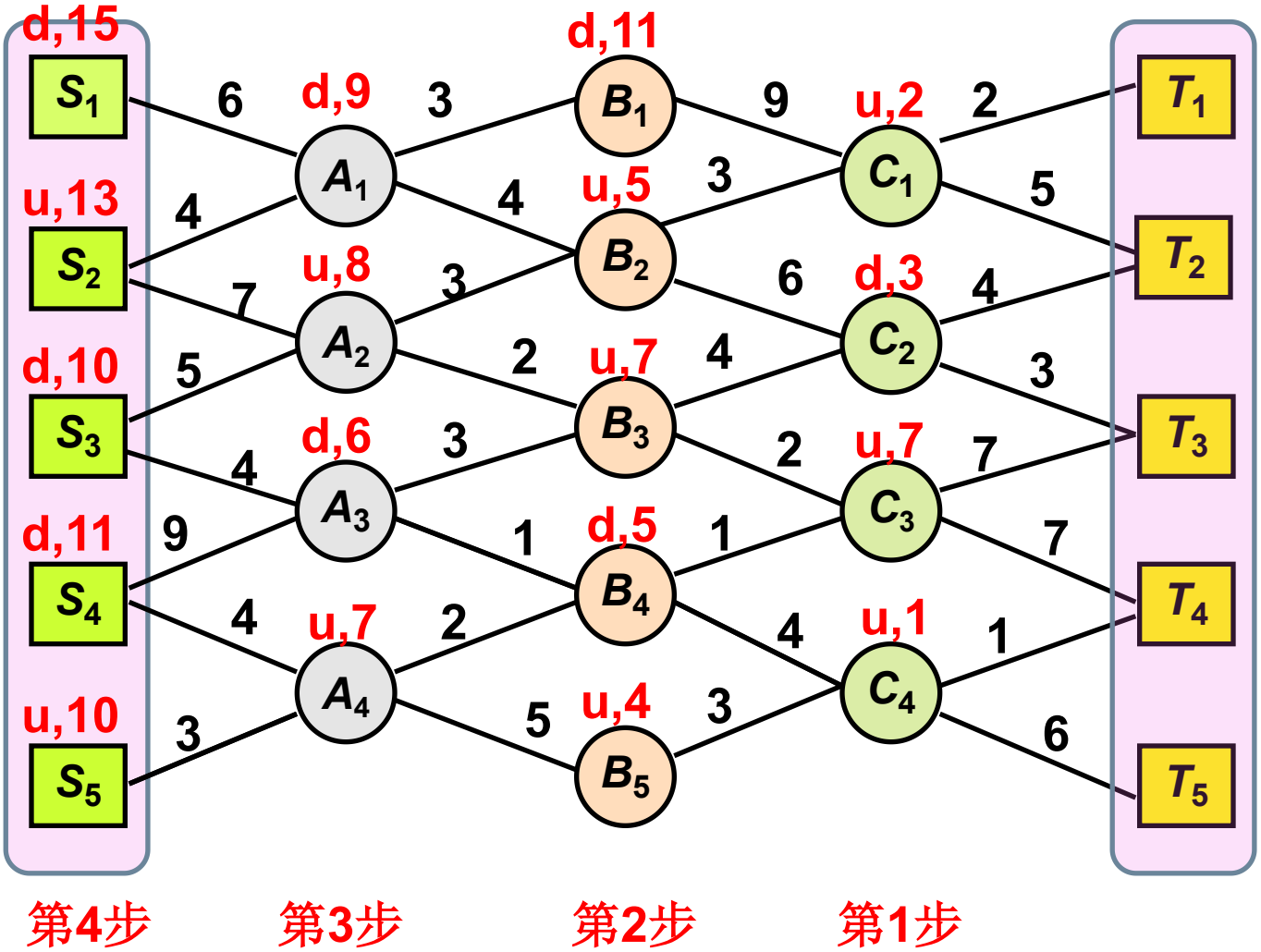
动态规划算法求解过程：第2步(down，up)



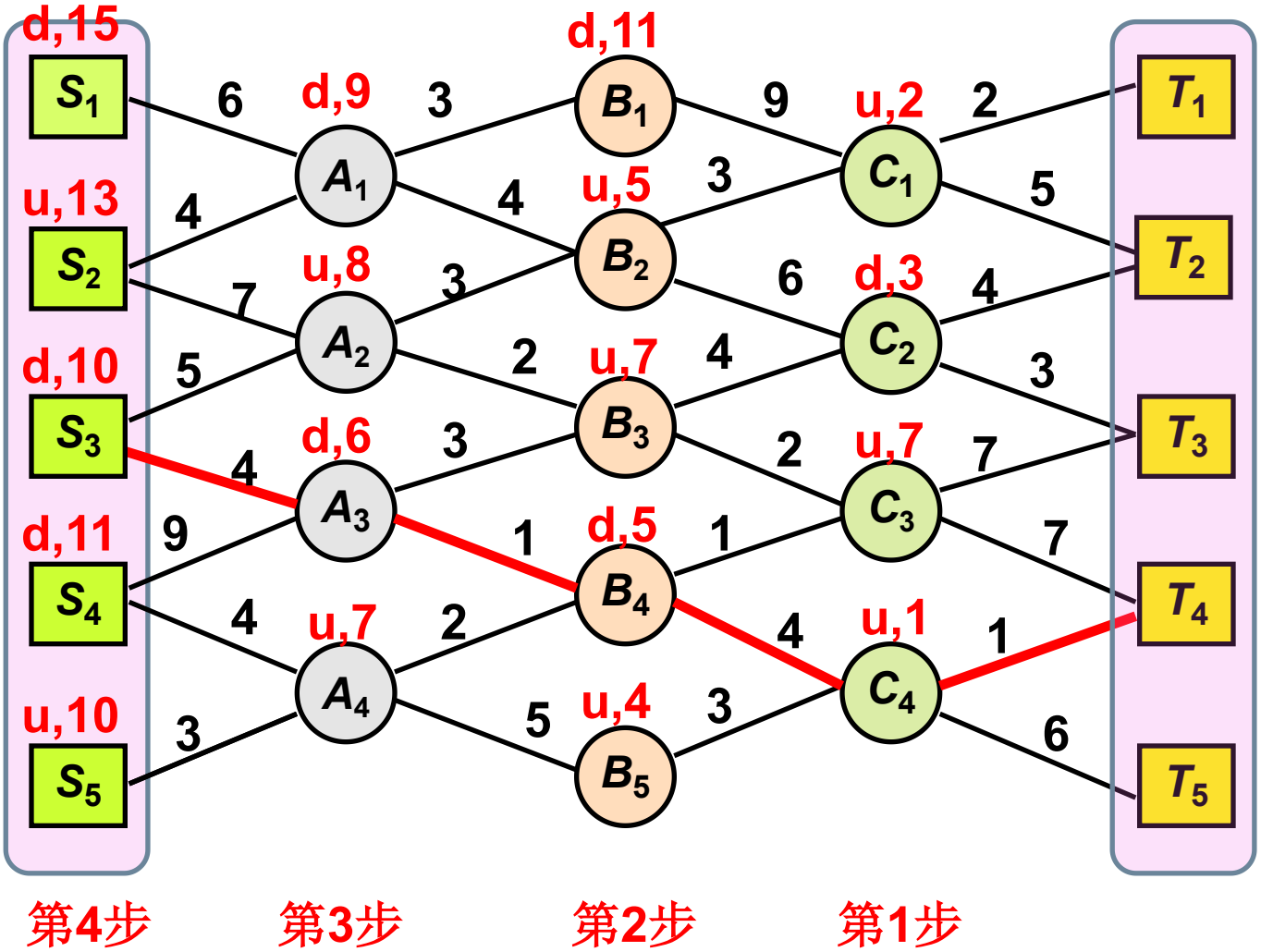
动态规划算法求解过程：第3步(down，up)



动态规划算法求解过程：第4步(down，up)

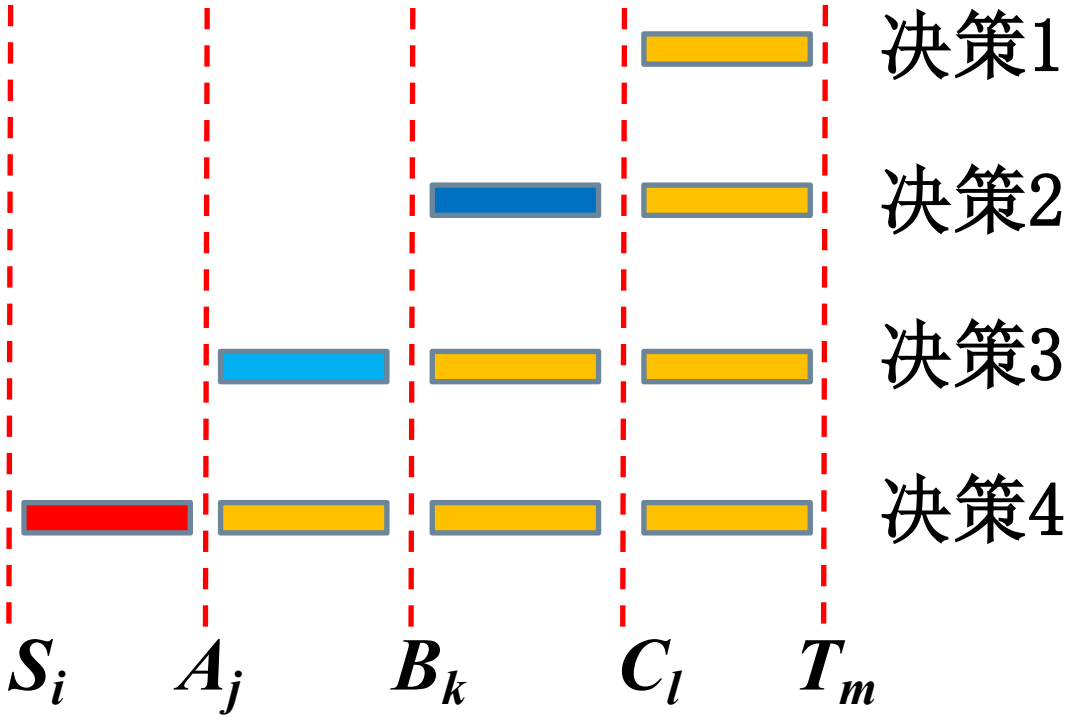


动态规划算法求解过程：找路径(down，up)



分析：子问题界定

后边界不变，前边界前移



动态规划的算法复杂度

加法运算次数：每一层$2n$次加法　$O(n)$

共有k层　$O(nk)$

比较运算次数：每一层比较n次　$O(n)$

共有k层　$O(nk)$

所以，动态规划的算法复杂度$O(nk)$

前边的蛮力算法是算法复杂度为$O(nk2^k)$

一、动态规划的基本思想

最短路长的依赖关系(判断序列)

$$\boldsymbol{F}(C_l)=\min_m\{C_lT_m\}\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\text{决策1}$$

$$\boldsymbol{F}(B_k)=\min_l\{B_kC_l+\boldsymbol{F}(C_l)\}\cdots\cdots\cdots\cdots\text{决策2}$$

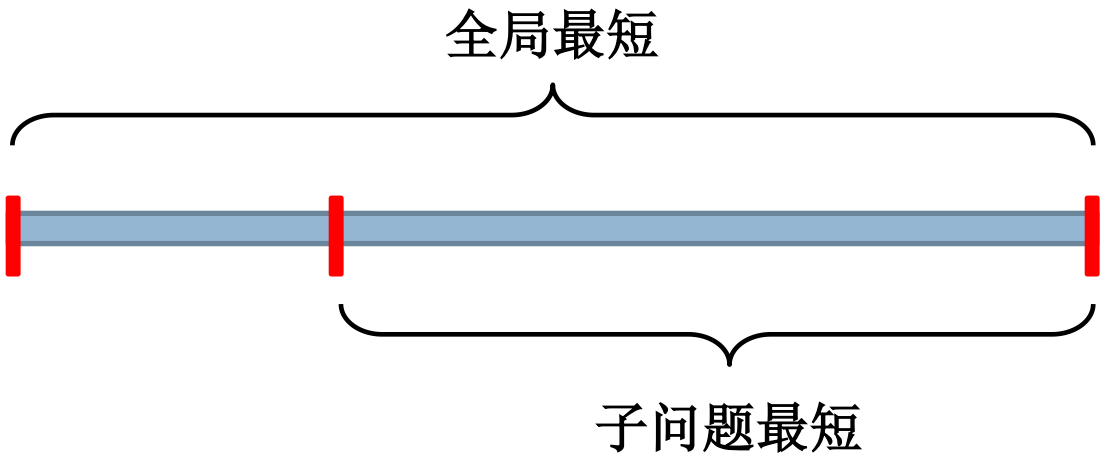
$$\boldsymbol{F}(A_j)=\min_k\{A_jB_k+\boldsymbol{F}(B_k)\}\cdots\cdots\cdots\cdots\text{决策3}$$

$$\boldsymbol{F}(S_i)=\min_j\{S_iA_j+\boldsymbol{F}(A_j)\}\cdots\cdots\cdots\cdots\text{决策4}$$

优化函数值之间存在依赖关系

最优子结构性质(即优化原则)

优化函数的特点：任何最短路的子路径相对于子问题始、终点最短



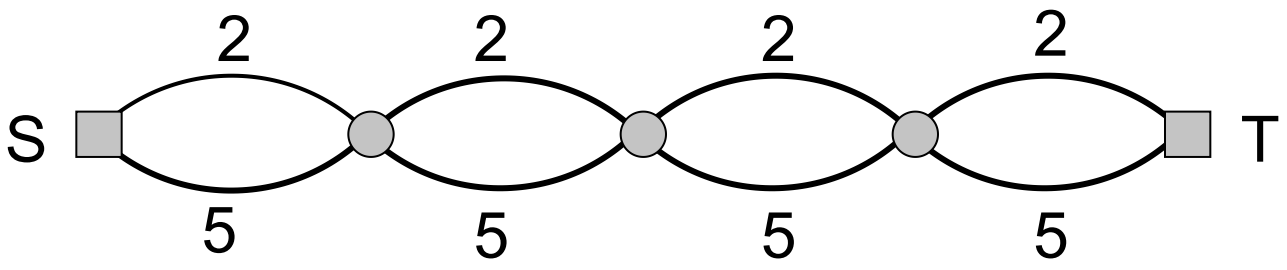
求解步骤：先确定子问题的边界，再从最小的子问题开始，进行多步判断

二、使用动态规划技术的条件--优化原则：

一个最优决策序列的任何子序列本身一定是相对于子序列的初始和结束状态的最优决策序列。

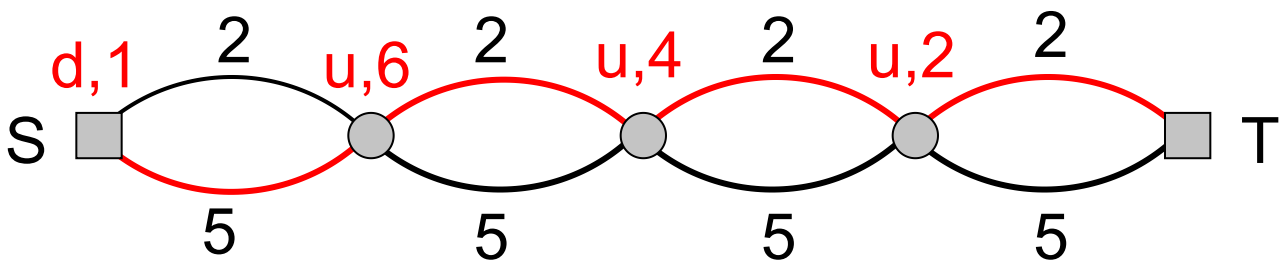
优化原则的反例-不符合优化原则的问题，则不能使用动态规划算法。

例3.2 求总长模10的最小路径



优化原则的反例

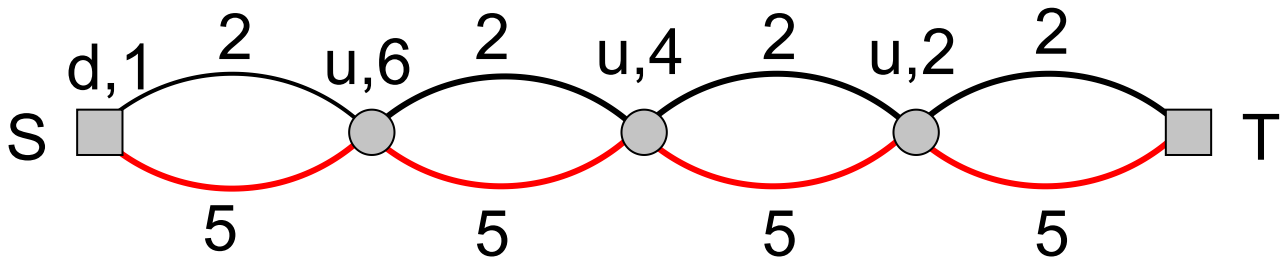
例3.2 求总长模10的最小路径



动态规划算法的解：下、上、上、上

优化原则的反例

例3.2 求总长模10的最小路径



动态规划算法的解：下、上、上、上

真正的最优解：下、下、下、下

不满足优化原则，不能使用动态规划设计技术

3.2 算法设计步骤

1. 问题建模，优化的目标函数是什么？约束条件是什么？

2. 如何划分子问题（边界）

3. 是否满足优化原则？

4. 存在着什么依赖关系（递推方程）？

5. 最小子问题怎样界定？其优化函数值，即初值等于什么？

例3.3 矩阵序列乘法问题

设A_1，A_2，…，A_n为矩阵序列，A_i 为 $P_{i-1}\times P_i$ 阶矩阵，$i = 1, 2, \cdots, n$。试确定矩阵的乘法顺序使得元素相乘的总次数最少。

输入：向量 $P = \langle P_0, P_1, \cdots, P_n \rangle$，$n$个矩阵的行数、列数

输出：矩阵链乘法加括号的位置

矩阵相乘的基本运算次数：

矩阵A：i 行j 列，B：j 行k 列，以元素相乘做基本运算，计算AB 的工作量

$$\begin{bmatrix} \cdots & & \\ a_{t1} & a_{t2} & \cdots a_{tj} \\ \cdots & & \\ \cdots & & \end{bmatrix} \times \begin{bmatrix} \cdots b_{1s} \cdots \\ \cdots b_{2s} \cdots \\ \vdots \\ \cdots b_{js} \cdots \end{bmatrix} = \begin{bmatrix} \cdots\cdots\cdots \\ \cdots c_{ts} \cdots \\ \vdots \\ \cdots\cdots\cdots \end{bmatrix}$$

$$c_{ts} = a_{t1}b_{1s} + a_{t2}b_{2s} + \cdots + a_{tj}b_{js}$$

AB：i 行k 列，计算每个元素需要做j 次乘法，总计乘法数为$i \times j \times k$ 次。

实例：$\boldsymbol{P} = \langle 10,\ 100,\ 5,\ 50 \rangle$

$$\boldsymbol{A}_1:\ 10 \times 100,\quad \boldsymbol{A}_2:\ 100 \times 5,\quad \boldsymbol{A}_3:\ 5 \times 50,$$

乘法次序

$$(\boldsymbol{A}_1\ \boldsymbol{A}_2)\boldsymbol{A}_3:\ 10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500$$

$$\boldsymbol{A}_1(\boldsymbol{A}_2\ \boldsymbol{A}_3):\ 10 \times 100 \times 50 + 100 \times 5 \times 50 = 75000$$

一、蛮力算法(穷举)：搜索空间的规模

加 n 对括号的方法有 $\frac{1}{n+1}\binom{2n}{n}$ 种，$2n$中取n的组合数，是一个Catalan数，是指数级别。代入Stirling公式，得：

$$W(n)=\Omega\left(\frac{1}{n+1}\frac{(2n)!}{n!\,n!}\right)=\Omega\left(\frac{1}{n+1}\frac{\sqrt{2\pi 2n}\left(\frac{2n}{e}\right)^{2n}}{\sqrt{2\pi n}\left(\frac{n}{e}\right)^{n}\sqrt{2\pi n}\left(\frac{n}{e}\right)^{n}}\right)$$

$$=\Omega\left(\frac{1}{n+1}\frac{n^{\frac{1}{2}}2^{2n}n^{2n}e^{n}e^{n}}{e^{2n}n^{\frac{1}{2}}n^{n}n^{\frac{1}{2}}n^{n}}\right)=\Omega(2^{2n}/n^{\frac{3}{2}})$$

每种组合需要$O(n)$次乘法，算法时间复杂度$\Omega(2^{2n}/n^{0.5})$

二、动态规划算法

1. 由i 和j 确定子问题的边界：输入$P = \langle P_0, P_1, \cdots, P_n \rangle$，$A_{i..j}$ 表示乘积$A_i A_{i+1} \cdots A_j$ 的结果，其最后一次相乘是$A_{i..j} = A_{i..k}A_{k+1..j}$，$k$为划分的位置。

动态规划方法：将求$A_{i..j}$最优解$m[i, j]$，变成先求$A_{i..k}$和$A_{k+1..j}$最优解$m[i, k]$和$m[k+1, j]$。

此问题符合优化原则吗？

证明：当$A_{i..j}$的解$m[i,j]$最小时，子问题的解$m[i,k]$和$m[k+1,j]$也是最小的。//证明此问题符合优化原则

反证法：如果$m[i,j]$最小时，存在更小的$m'[i,k]$，

已知 $m[i,j]= m[i,k]+m[k+1,j]+P_{i-1}P_kP_j$

设 $m'[i,j]= m'[i,k]+m[k+1,j]+P_{i-1}P_kP_j$

∵ $m'[i,k]$ 比 $m[i,k]$更小

∴ $m'[i,j]$会比$m[i,j]$更小，这与$m[i,j]$最小矛盾，

证明完毕。

2. 确定优化函数和递推方程：

$m[i, j]$ 表示得到 $A_{i..j}$ 的最少的相乘次数，则递推方程和初值

$$m[i,j]=\begin{cases}0 & i=j\\ \min\limits_{i\le k<j}\{m[i,k]+m[k+1,j]+P_{i-1}P_kP_j\} & i<j\end{cases}$$

$$A_{i\cdots k}A_{k+1\cdots j} \Rightarrow \underbrace{\boxed{A_i \cdots A_k}}_{P_{i-1}\times P_k}\underbrace{\boxed{A_{k+1} \cdots A_j}}_{P_k\times P_j} \Rightarrow P_{i-1}\times P_k\times P_j$$

3. 设立标记函数：为了确定加括号的次序，设计表 $s[i,j]$，记录求得最优时最后一次运算的划分位置k。

例如：$m[2,7]=30$，$s[2,7]=5$ ， 即$i=2$，$j=7$，$k=5$

表示$A_2 \cdots A_7$链的最优乘法次数30，对应的最优划分括号位置在5处(或者说最后一次相乘发生在5处)，即$A_{2\ldots5}A_{6\ldots7}$。

三、算法递归实现(可以忽略不讲，两个变量不适用主定理)

算法3.1 RecurMatrixChain（P, i, j）

```
if i==j then return m[i, j]=0
m[i, j]←∞
s[i, j]←i
for k←i to j−1 do
    q ← RecurMatrixChain( P, i, k )
        + RecurMatrixChain(P, k+1, j ) + p_{i−1} p_k p_j
    if q < m[i, j] then
        m[i, j]←q
        s[i, j]←k
return m[i, j]
```

递归实现的复杂性分析：

复杂性满足递推关系

$$T(n) \geq \begin{cases} O(1) & n = 1 \\ \sum_{k=1}^{n-1}(T(k) + T(n-k) + O(1)) & n > 1 \end{cases}$$

$$T(n) \geq \Theta(n) + \sum_{k=1}^{n-1} T(k) + \sum_{k=1}^{n-1} T(n-k)$$

$$\Theta(n) + \sum_{k=1}^{n-1} T(k) + \sum_{k=1}^{n-1} T(n-k)$$

$$= \Theta(n) + \sum_{k=1}^{n-1} T(k) + \sum_{h=1}^{n-1} T(h) \cdots\cdots k = n - h$$

$$= \Theta(n) + 2\sum_{k=1}^{n-1} T(k)$$

补充知识：数学归纳法

叙述一个描述算法正确性的命题$P(n)$，n为算法步数或者问题规模

①归纳基础：$P(1)$ 或 $P(n_0)$为真，n_0为某个自然数

②归纳步骤：$P(k) \Rightarrow P(k+1)$　　第一数学归纳法

$\forall k(k<n)P(k) \Rightarrow P(n)$　　第二数学归纳法

归纳法1：$P(1) \Rightarrow P(2) \Rightarrow P(3) \Rightarrow P(4) \cdots$

归纳法2：

$$\begin{array}{ccccccc} & & & & P(1) & & \\ & & P(1) & & P(2) & & \\ P(1) & \Rightarrow & P(2) & \Rightarrow & P(3) & \Rightarrow & P(4) \cdots \end{array}$$

定理：对于$n>1$，$T(n)=\Omega(2^{n-1})$

应用数学归纳法2证明：

①$n=2$，$T(2)\geq c = c_1 2^{2-1}$。 命题成立，$c_1=c/2$为某个正数

②假设对于任何n大于2且小于的k命题为真，即 $T(k)\geq c_1 2^{k-1}$

则对于$n>k$，存在c'使得

$$T(n)\geq c'n+2\sum_{k=1}^{n-1}T(k)\geq c'n+2\sum_{k=2}^{n-1}c_1 2^{k-1}$$

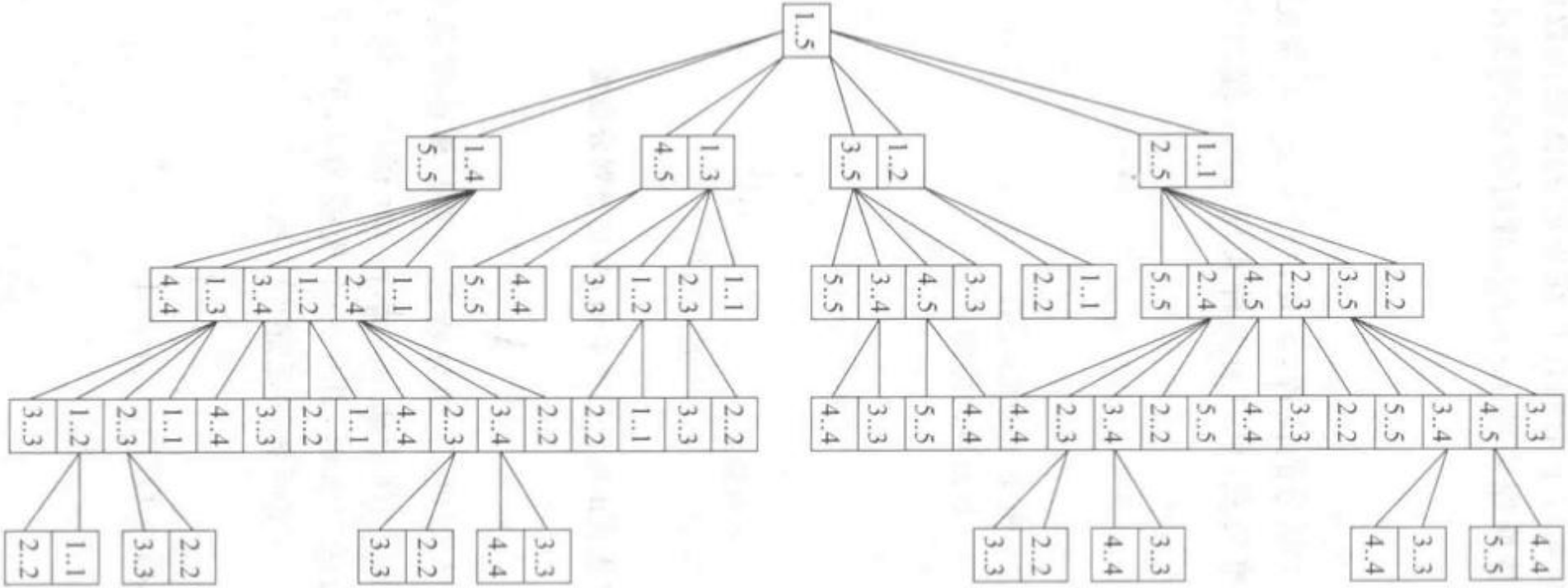
$$=c'n+2c_1\frac{2^{(n-1)-1}\times 2-2^{2-1}}{2-1}$$

k=1时$T(1)=0$

命题亦成立 $$=c'n+c_1(2^n-4)\geq c_1 2^{n-1}$$

所以，**动态规划问题递归算法的复杂性$T(n)=\Omega(2^n)$**

n=5时，子问题的划分：课本上的递归算法子问题树

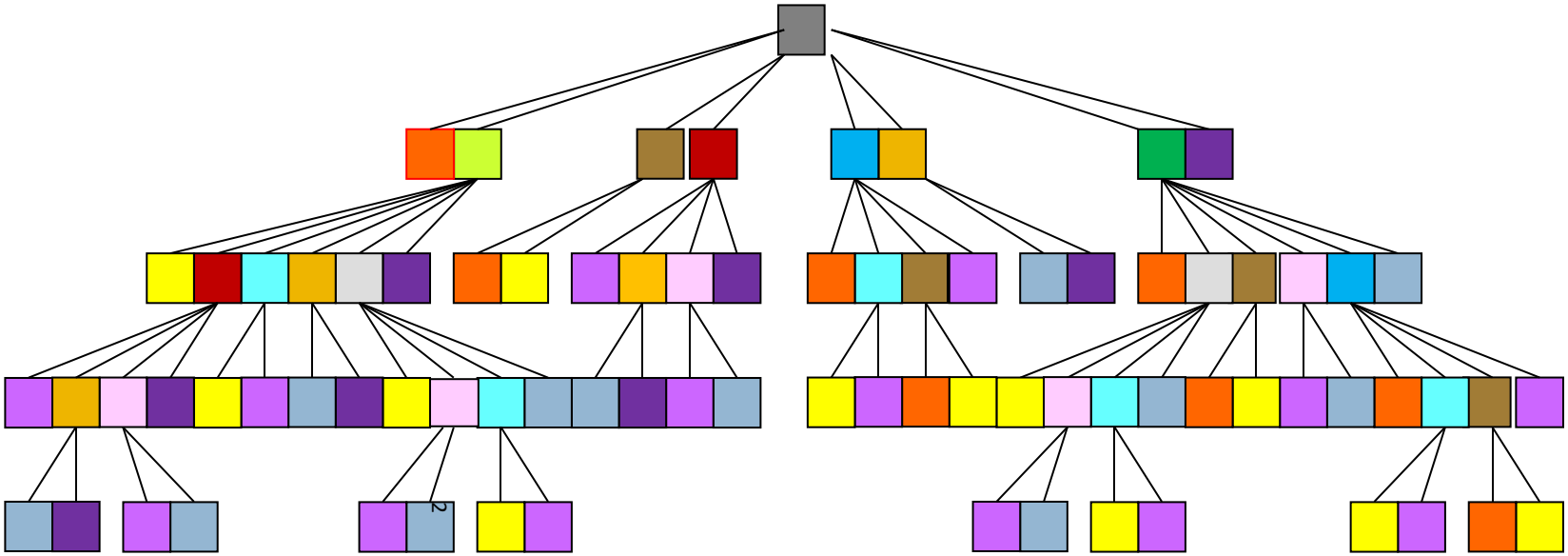


P58图3.4 子问题

递归算法复杂性高的原因：子问题重复计算

n=5，计算子问题：81个；不同的子问题：15个

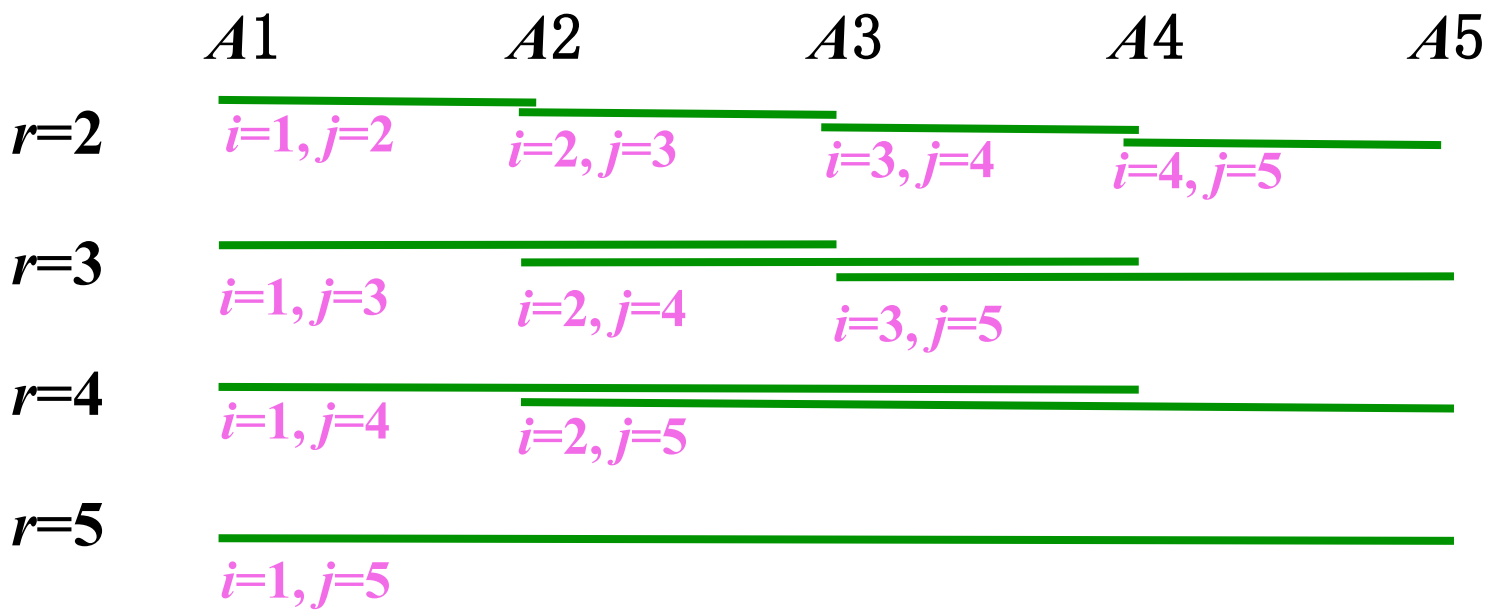
子问题	1-1	2-2	3-3	4-4	5-5	1-2	2-3	3-4	4-5	1-3	2-4	3-5	1-4	2-5	1-5
数	8	12	14	12	8	4	5	5	4	2	2	2	1	1	1



三、算法迭代实现

连城矩阵个数　子问题起始位　子问题k的位置

n=5的迭代过程(r第1层循环2~n、i第2层循环1~$n-r+1$、 k第3层$i+1$~$j-1$循环($j=i+r-1$)，求$m[i,j]$、$s[i,j]$)



算法3.2 MatrixChain(P, n)

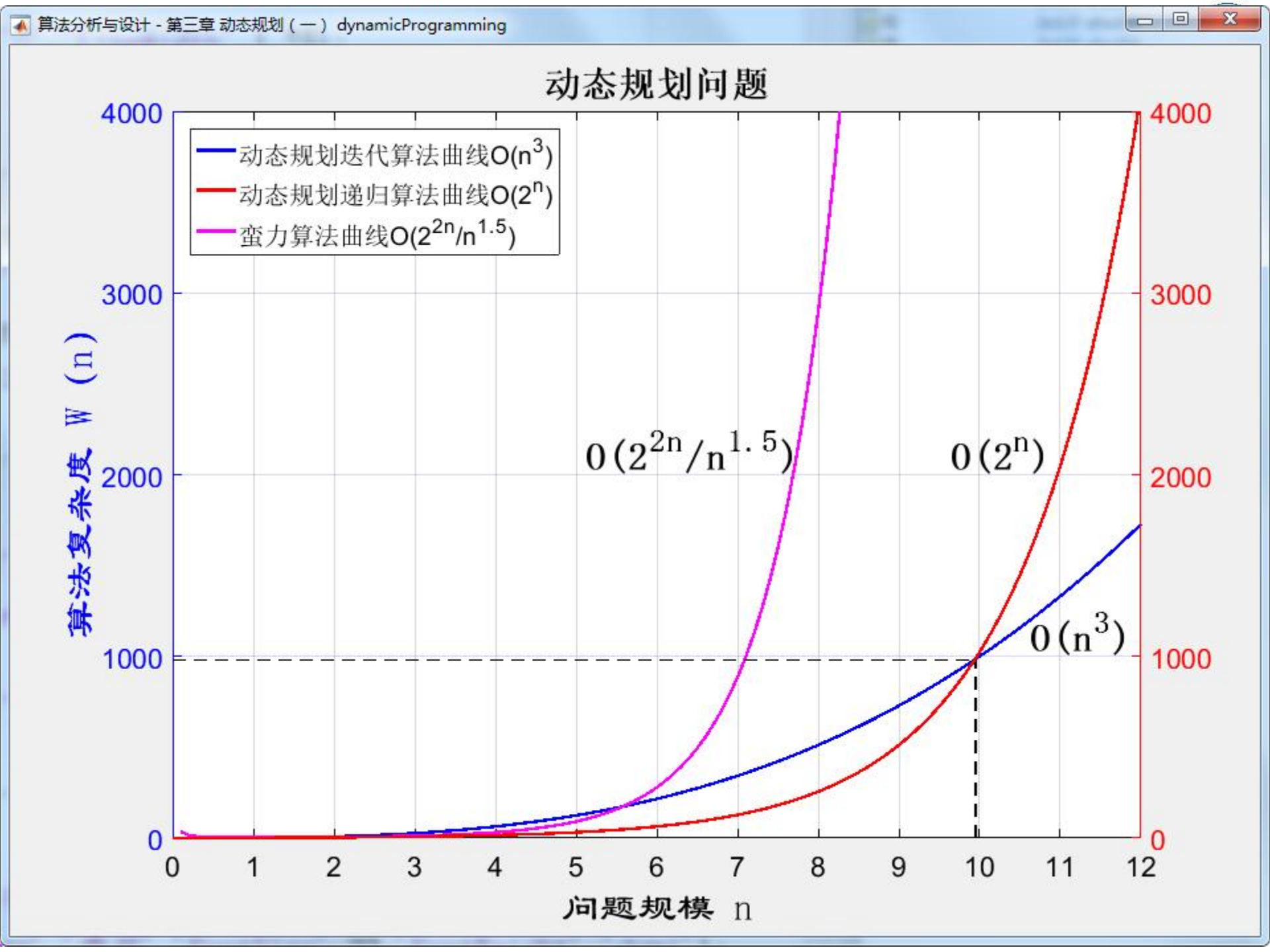
```
令所有的 m[i,i]初值为0，s[i,j]初值为i    1≤ i ≤ j ≤ n
for  r ←2  to  n  do                    // r为计算的矩阵链长度
    for i ←1  to  n−r+1  do             // n−r+1为最后r链的始位置
        j ← i +r−1                      // 计算链i—j，j 链尾
        m[i, j] ← m[i+1, j] + p_{i−1}*p_i*p_j // A_i(A_{i+1}..A_j)
        s[i, j] ← i                     //记录分割位置
        for  k ← i +1  to  j−1  do
            t ← m[i, k]+m[k+1,j]+p_{i−1}*p_k*p_j //(A_i..A_k)(A_{k+1}..A_j)
            if  t < m[i, j] then
                m[i, j]← t              //替换最优值
                s[i, j]← k              //记录对于最优分割位置
```

复杂性分析：

行2, 3, 7循环进行都是 $O(n)$，

循环内为 $O(1)$

所以，动态规划问题迭代算法复杂度$W(n)=O(n^3)$


算法分析与设计 - 第三章 动态规划（一） dynamicProgramming
动态规划问题
动态规划迭代算法曲线O(n^3)
动态规划递归算法曲线O(2^n)
蛮力算法曲线O(2^2n/n^1.5)
O(2^2n/n^1.5)
O(2^n)
O(n^3)
算法复杂度 W（n）
问题规模 n
0
1000
2000
3000
4000
0
1
2
3
4
5
6
7
8
9
10
11
12

实例

输入 P = ⟨30, 35, 15, 5, 10, 20⟩, n=5,

矩阵链：A_1 A_2 A_3 A_4 A_5，其中

A_1:30×35，A_2:35×15，A_3:15×5，A_4:5×10，A_5:10×20

备忘录

r=1	m[1,1]=0	m[2,2]=0	m[3,3]=0	m[4,4]=0	m[5,5]=0
r=2	m[1,2]=15750	m[2,3]=2625	m[3,4]=750	m[4,5]=1000	
r=3	m[1,3]=7875	m[2,4]=4375	m[3,5]=2500		
r=4	m[1,4]=9375	m[2,5]=7125			
r=5	m[1,5]=11875				

r=2	s[1,2]=1	s[2,3]=2	s[3,4]=3	s[4,5]=4	
r=3	s[1,3]=1	s[2,4]=3	s[3,5]=3		
r=4	s[1,4]=3	s[2,5]=3			
r=5	s[1,5]=3				

r=1	m[1,1]=0	m[2,2]=0	m[3,3]=0	m[4,4]=0	m[5,5]=0
r=2	m[1,2]=15750	m[2,3]=2625	m[3,4]=750	m[4,5]=1000	
r=3	m[1,3]=7875	m[2,4]=4375	m[3,5]=2500		
r=4	m[1,4]=9375	m[2,5]=7125			
r=5	m[1,5]=11875				

r=2	s[1,2]=1	s[2,3]=2	s[3,4]=3	s[4,5]=4	
r=3	s[1,3]=1	s[2,4]=3	s[3,5]=3		
r=4	s[1,4]=3	s[2,5]=3			
r=5	s[1,5]=3				

解的追踪

$s[1,5]=3$ ➔ $(A_1A_2A_3)(A_4A_5)$

$s[1,3]=1$ ➔ $A_1(A_2A_3)$

输出结论为：计算顺序 $(A_1(A_2A_3))(A_4A_5)$

最小的乘法次数 11875

矩阵相乘问题小结

1. 动态规划的基本思想

把复杂问题拆分成一步一步解决的子问题序列，由简至难顺序解决各个子问题，前边问题的解决有助于后边的问题解决，从而简化算法的复杂性。

2. 使用动态规划技术的条件:优化原则

(1) 最优子结构性质：一个最优决策序列的任何子序列本身一定是相对于子序列的初始和结束状态的最优决策序列。此为关键因素。

(2) 重叠子问题：前边的子问题解有助于后面问题的解决。此因素决定能简化多少工作量。

矩阵相乘问题小结

3. 动态规划算法的复杂度

时间复杂度$O(n^3)$ — 矩阵相乘问题

4. 动态规划算法设计步骤

①划分子问题　**用参数表达子问题的边界，将问题求解转变成多步判断的过程。**

②确定优化函数　**以该函数的极大(或极小)作为判断的依据，确定是否满足优化原则。**

③列出关于优化函数的递推方程（或不等式)和边界条件

④考虑是否需要设立标记函数

⑤自底向上计算，以备忘录方法（表格)存储中间结果

习题3 P83 3.1

3.1 用动态规划算法求解下列组合优化问题

$$\max\{g_1(x_1)+g_2(x_2)+g_3(x_3)\}$$

$$x_1^2+x_2^2+x_3^2\le 10$$

x_1，x_2，x_3为非负整数

其中函数$g_1(x)$，$g_2(x)$，$g_3(x)$的值如下表：

x	$g_1(x)$	$g_2(x)$	$g_3(x)$	x	$g_1(x)$	$g_2(x)$	$g_3(x)$
0	2	5	8	2	7	16	17
1	4	10	12	3	11	20	22

结 束

谢谢收看，再见！